# Practical - 1

## Implementation and Time analysis of sorting algorithms

a. Bubble Sort
b. Selection Sort
c. Insertion Sort
d. Merge Sort
e. Quick Sort

## a. Bubble sort

### Introduction

Bubble Sort is the simplest sorting algorithm. It repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest element moves to its correct position at the end of the array.

### Aim

To sort a list of elements in ascending order using the Bubble Sort algorithm.

### Objective

- To understand the working of Bubble Sort.
- To arrange data in ascending order.
- To learn element comparison and swapping.

### Algorithm

1. Read the number of elements.
2. Store the elements in an array.
3. Compare each pair of adjacent elements.
4. Swap them if they are in the wrong order.
5. Repeat until the array is sorted.
6. Display the sorted array.

### Pseudocode

START
Read n
Read array A
FOR i = 0 to n-2
  FOR j = 0 to n-i-2
    IF A[j] > A[j+1]
      Swap A[j] and A[j+1]
    ENDIF
  ENDFOR
ENDFOR

Print A
STOP
Code:

Implementation and Time analysis of Algorithms

1.Bubble Sort

[8]:
```
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:")
print(arr)
```

```
Enter the number of elements: 5
Enter the elements:
8
9
8
64
5
Sorted array:
[5, 8, 8, 9, 64]
```

Advantages

- Easy to understand and implement.
- Works well for small datasets.
  Requires no extra memory.

Disadvantages

- Slow for large datasets.
- Large number of comparisons and swaps. Not suitable for real-world applications.

Conclusion

Bubble Sort is a simple algorithm suitable for learning sorting concepts, but it is inefficient for large datasets.

## 2. Selection Sort

Introduction

Selection Sort repeatedly selects the smallest element from the unsorted portion of the array and places it at the correct position.

Aim

To sort elements using the Selection Sort algorithm.

Objective

- To learn minimum element selection.
- To understand in-place sorting.
- To arrange data efficiently for small datasets.

Algorithm

1. Read the array.
2. Find the smallest element.
3. Swap it with the first unsorted element.
4. Repeat until the array is sorted.
5. Print the sorted array.

Pseudocode

```
START
Read n
Read array A

FOR i = 0 to n-1
min = i
   FOR j = i+1 to n-1
      IF A[j] < A[min]
   min = j ENDIF
   ENDFOR

   Swap A[i] and A[min]
ENDFOR

Print A
STOP
```

Code:

2. Selection Sort

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
for i in range(n):
    min_index = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j
    arr[i], arr[min_index] = arr[min_index], arr[i]
print("Sorted array:")
print(arr)
```

```
Enter the number of elements: 5
Enter the elements:
9
5
6
7
8
Sorted array:
[5, 6, 7, 8, 9]
```

Advantages

- Easy to implement.
- Requires less memory.
- Performs fewer swaps than Bubble Sort.

Disadvantages

Time complexity is $O(n^2)$.

- Inefficient for large datasets.
- Does not preserve the order of equal elements.

Conclusion

Selection Sort is simple and performs fewer swaps, but its quadratic time complexity makes it unsuitable for large datasets.

## 3. Insertion Sort

Introduction

Insertion Sort builds the sorted array one element at a time by inserting each element into its proper position.

Aim

To sort elements using the Insertion Sort algorithm.

Objective

- To understand insertion-based sorting.
- To sort data efficiently for small datasets. □ To learn shifting of elements.

Algorithm

1. Read the array.
2. Assume the first element is sorted.
3. Pick the next element.
4. Insert it into the correct position.
5. Repeat until all elements are sorted.
6. Display the sorted array.

Pseudocode

```
START
Read n
Read array A

FOR i = 1 to n-1
   key = A[i]
   j = i-1

   WHILE j >= 0 AND A[j] > key
      A[j+1] = A[j]
      j = j-1
   ENDWHILE

   A[j+1] = key
```

ENDFOR

Print A
STOP

Code:

3. Insertion Sort

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
for i in range(1, n):
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = key

print("Sorted array:")
print(arr)
```

```
Enter the number of elements: 5
Enter the elements:
9
5
2
8
3
Sorted array:
[2, 3, 5, 8, 9]
```

Advantages

- Efficient for small datasets.
- Stable sorting algorithm.
  Works well for nearly sorted data.

Disadvantages

- Slow for large datasets.
- Time complexity is $O(n^2)$ in average and worst cases.

Conclusion

Insertion Sort is efficient for small and nearly sorted datasets but becomes slow as the dataset size increases.

---

## 4. Merge Sort

Introduction

Merge Sort is a Divide and Conquer algorithm. It divides the array into smaller subarrays, sorts them recursively, and merges them into a sorted array.

Aim

To sort elements using the Merge Sort algorithm.

Objective

- To understand Divide and Conquer.
- To efficiently sort large datasets.
- To learn recursive sorting techniques.

Algorithm

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves.
4. Repeat until the complete array is sorted.
5. Display the sorted array.

Pseudocode

```
START
MergeSort(A)

IF length(A) > 1
   Divide A into Left and Right
```

MergeSort(Left)
MergeSort(Right)

Merge Left and Right into A
ENDIF

Print A
STOP

Code:

4. Merge Sort

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
```

```
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

merge_sort(arr)

print("Sorted array:")
print(arr)
```

```
Enter the number of elements: 5
Enter the elements:
5
2
3
9
1
Sorted array:
[1, 2, 3, 5, 9]
```

Advantages

- Fast and efficient.
  Stable sorting algorithm.
- Guaranteed O(n log n) time complexity.

Disadvantages

- Requires extra memory.
- Recursive implementation is more complex.

Conclusion

Merge Sort is one of the most efficient sorting algorithms for large datasets and guarantees consistent performance.

---

5. Quick Sort

Introduction

Quick Sort is a Divide and Conquer algorithm that selects a pivot element and partitions the array into smaller and larger elements before recursively sorting them.

Aim

To sort elements using the Quick Sort algorithm.

Objective

- To understand partitioning.
- To efficiently sort large datasets. □ To implement recursive sorting.

Algorithm

1. Choose a pivot element.
2. Partition the array.
3. Place the pivot in its correct position.
4. Recursively sort left and right subarrays.
5. Display the sorted array.

Pseudocode

START
QuickSort(A, low, high)

IF low < high pivot =
Partition(A)

QuickSort(A, low, pivot-1)
QuickSort(A, pivot+1, high)
ENDIF

Print A
STOP

Code:

```
]: def partition(arr, low, high):
       pivot = arr[high]
       i = low - 1

       for j in range(low, high):
           if arr[j] <= pivot:
               i += 1
               arr[i], arr[j] = arr[j], arr[i]

       arr[i + 1], arr[high] = arr[high], arr[i + 1]
       return i + 1

   def quick_sort(arr, low, high):
       if low < high:
           pi = partition(arr, low, high)

           quick_sort(arr, low, pi - 1)
           quick_sort(arr, pi + 1, high)

   n = int(input("Enter the number of elements: "))
   arr = []

   print("Enter the elements:")
   for i in range(n):
       arr.append(int(input()))

   quick_sort(arr, 0, n - 1)

   print("Sorted array:")
   print(arr)
```

```
Enter the number of elements: 5
Enter the elements:
9
8
7
6
5
Sorted array:
[5, 6, 7, 8, 9]
```

Advantages

- Very fast in practice.
- Average time complexity is O(n log n).
  Requires less memory than Merge Sort.

Disadvantages

- Worst-case time complexity is $O(n^2)$.
- Recursive implementation.
- Performance depends on pivot selection.

Conclusion

Quick Sort is one of the fastest sorting algorithms for practical applications. Although its worst-case complexity is $O(n^2)$, good pivot selection usually provides excellent average-case performance of O(n log n).

ThankYou